

Examen réparti

Carlos Agon

Déroulement de L'examen

Tous les documents sont autorisés, et notamment ceux du cours.

L'utilisation de tout moyen de communication (téléphone, tablette, ordinateur personnel, etc.) est interdite.

L'utilisation d'une clé ou disque USB personnel, en lecture seule et étiqueté à votre nom, est autorisée ; vous ne devez pas le prêter à un camarade.

Le barème pour chaque question n'est donné qu'à titre indicatif et pourra être adapté lors de la correction. **L'examen dure 2h.** Votre copie sera formée de fichiers textuels que vous laisserez aux endroits spécifiés dans votre espace de travail pour Eclipse. L'espace de travail pour Eclipse sera obligatoirement nommé `workspace` et devra être un sous-répertoire direct de votre répertoire personnel.

Le langage à étendre est ILP2. Le packaging Java correspondant sera nommé `com.paracamplus.ilp2.ilp2partiel`. Sera ramassé, à partir de votre `~/workspace` (situé sous ce nom directement dans votre HOME), le **seul** répertoire `~/workspace/ILP_SU-master/Java/src/com/paracamplus/ilp2/ilp2partiel/` et tous ses sous-répertoires.

L'examen sera corrigé à la main, il est donc absolument inutile de s'acharner sur un problème de compilation ou sur des méthodes à contenu informatif faible. Il est beaucoup plus important de rendre aisé, voire plaisant, le travail du correcteur et de lui indiquer, par tout moyen à votre convenance, de manière claire, compréhensible et terminologiquement précise, comment vous surmontez cette épreuve.

Mise en route

- Ouvrez Eclipse
- Choisissez le repertoire `workspace` comme `workspace`
- (facultatif, puisqu'on ne demande pas le code du compilateur) Compilez la bibliothèque d'exécution en faisant :

```
cd ~/workspace/ILP_SU-master/C/; make
```

- le plugin ANTLR devrait fonctionner sans réglages particuliers. Toutefois, si ce n'est pas le cas, vous pouvez toujours utiliser ANTLR en ligne de commande à l'aide de l'archive fournie dans les sources d'ILP, en faisant :

```
cd ~/workspace/ILP_SU-master/ANTLRGrammars
java -jar ~/workspace/ILP_SU-master/Java/jars/antlr-4.4-complete.jar \
-lib ~/workspace/ILP_SU-master/ANTLRGrammars \
-o ~/workspace/ILP_SU-master/target/generated-sources/antlr4/ \
LE_NOM_DE_VOTRE_GRAMMAIRE.g4
```

On rappelle que l'on peut aussi lancer le plug-in à la main depuis Eclipse avec "Run as..." si il ne se lance pas automatiquement lors de l'édition d'une grammaire.

Dépannage

- Si jamais eclipse se bloque en essayant de se connecter à svn, tapez dans un terminal :

```
killall eclipse; killall java
```

puis relancez eclipse.

Introduction

Dans cet examen, nous voulons étendre ILP2 avec deux nouvelles expressions `freeze` et `frozen`.

`freeze` prend une fonction et un ensemble d'arguments et invoque la fonction avec les arguments puis stocke le résultat en interne. Ce résultat stocké est renvoyé chaque fois que `frozen` est appelée avec la même fonction. Dans les deux cas, `freeze` et `frozen`, la fonction est donnée sous la forme d'une expression qui correspond à une fonction définie par le programme (on ne peut pas faire `freeze` et `frozen` sur de fonctions primitives).

Voici un exemple qui illustre cette extension :

```
function test(x,y)
  ( print "<->";
    x*y;
  );

print (test (4,2));
print (freeze (test, 5, 2));
print (frozen (test));
print (freeze (test, 7, 2));
print (frozen (test));
test (8,2);
```

Le résultat du programme précédent est 16, mais en plus il aura un effet de bord qui imprimera l'information suivante : "<->8<->1010<->1414<->".

- `test (4,2)` invoque `test`, imprime "<->" et retourne 8.
- `freeze (test, 5, 2)` invoque `test`, imprime "<->", retourne 10 et garde ce résultat.
- `frozen (test)` retourne 10.
- `freeze (test, 7, 2)` invoque `test`, imprime "<->", retourne 14 et garde ce nouveau résultat.
- `frozen (test)` retourne 14.
- `test (8,2)` invoque `test`, imprime "<->" et retourne 16.

Un appel à `frozen` sur une fonction qui n'a pas été précédée d'un appel à `freeze` sur cette même fonction doit signaler une erreur. Notez aussi qu'une invocation classique ne change pas la valeur éventuellement stockée par un `freeze`, cette dernière valeur peut être uniquement écrasée par un autre `freeze`.

Et voici un dernier exemple :

```
function test(x,y)
  x*y;

function test1(x,y,z)
  x*y*z;

let fun = test1 in
(
  freeze (fun, 5, 2, 3);
  freeze (test, 2,2);
  frozen (test1);
)
```

Dans cet exemple, le résultat est 30. Notez bien que l'arité de `freeze` est variable, en fait elle dépend de l'arité de la fonction à invoquer. `freeze` ne peut donc pas être implantée comme une primitive (d'arité fixée en ILP). On voit aussi que pour le premier appel à `freeze` la fonction est

donnée par une expression (`fun`) tandis que pour `frozen` on passe son nom (`test1`), cela n'affecte pas le bon fonctionnement du programme. Finalement, notez que l'appel `freeze(test, 2, 2)` ne modifiera pas le résultat associé précédemment à `test1` : chaque fonction a son propre espace de stockage indépendant pour son résultat.

1 Grammaire (2 points)

Ecrivez une grammaire dans le fichier :

`com/paracampus/ilp2/ilp2partiel/grammars/ILPMLgrammarPartiel.g4`
en incluant des règles pour les expressions `freeze` et `frozen`.

2 AST (3 points)

Proposez une extension de l'arbre syntaxique pour gérer l'introduction de `freeze` et `frozen`. On rappelle que vous devez fournir les classes AST ainsi que les interfaces correspondantes dans `com.paracampus.ilp2.ilp2partiel.ast` et `com.paracampus.ilp2.ilp2partiel.interfaces`. On n'oubliera pas d'inclure la classe `ASTfactory` correspondante et son interface `IASTfactory` ainsi que la nouvelle interface `IASTvisitor`.

3 Analyse syntaxique (2 points)

Ecrivez dans le package `com.paracampus.ilp2.ilp2partiel.parser` les nouvelles classes `ILPMLParser` et `ILPMLListener` permettant de construire les nouveaux nœuds ajoutés au langage.

4 Interprétation (5 points)

Implantez un nouvel interpréteur dans une classe

`com.paracampus.ilp2.ilp2partiel.interpreter.Interpreter`.

5 Compilation (6 points)

5.1 Questions (2 points)

Répondez aux questions suivantes :

- listez les classes et interfaces que vous auriez à ajouter pour mettre en œuvre le compilateur, en précisant pour chacune les classes et interfaces qu'elles étendent ou implantent ;
- la gestion de l'erreur d'un appel à `frozen` sur une fonction qui n'a pas été précédé d'un appel à `freeze` sur cette même fonction peut-elle se faire statiquement ? Justifiez votre réponse.

5.2 Schéma de compilation (4 points)

Proposez les schémas de compilation pour les expressions `freeze` et `frozen` dans un fichier `Compiler.txt` dans le package `com.paracampus.ilp2.ilp2examen.compiler`. Si vous pensez que d'autres schémas de compilation doivent être modifiés, donnez leurs nouvelles définitions. Si vous croyez qu'il faut faire des changements à la bibliothèque d'exécution, ajoutez le code C nécessaire.

6 Tests (2 points)

Créez un répertoire `com/paracamplus/ilp2/ilp2partiel/SamplesPartiel/` dans lequel vous mettrez un ensemble de programmes pour tester votre implantation. Pour chaque programme expliquez l'erreur ou le comportement que vous testez.